TECHNICAL REQUIREMENTS DOCUMENT · v1.5

AI 1차 채용면접 자동화 플랫폼 — TRD (기술 요구사항 정의서)

PRD v1.6에 적은 기능을 어떤 기술로 어떻게 만들지

항목	내용
문서명	AI 1차 채용면접 자동화 플랫폼 TRD
문서 종류	TRD(기술 요구사항 정의서). 개발할 때 보는 문서다
버전 · 작성일	v1.5 · 2026-10-29
기준 PRD	통합 PRD v1.6 (merged_prd_v1.6). 요구사항 번호(ACC-, JOB-, APP-, INT-, QST-, COD-, EVL-, RPT-, ADM-, RES-, CHT-, AC-)는 PRD와 같다
상태	초안. 6장 모델 배치, 동시 면접 상한 N의 처음 값, 시간 목표 같은 설정값은 운영 서버에서 재 본 뒤 확정
작성 방침	PRD에 적은 "무엇을"은 되풀이하지 않고, "어떻게"(구조·숫자·주고받는 형식)만 적는다. 숫자(재시도 횟수, 시간 제한 등)는 14장 설정값 표 한 곳에 모은다
관련 문서	PRD v1.6 · 용어집(처음 보는 용어를 가나다순으로 풀어 둔 별도 문서) · TRD_변경이력(TRD 버전마다 바뀐 것과 그 이유) · 테이블 정의서(DB 칸 목록) · merged_prd_v1.1_변경이력 · TRD_검토_결정기록(v1.5에서 고친 이유)

목차

1. 개요 및 설계 원칙 3

1-1. 목적 및 범위 3

1-2. 설계 원칙 3

1-3. 범위 제약 (관리자 1명 기준) 3

2. 시스템 구성 4

2-1. 시스템 구성도 4

2-2. 서비스별 역할 4

2-3. 도커 배포 구성도 5

3. 기술 스택 및 버전 (Tech Stack) 6

4. 데이터 설계 (Data Structure) 8

4-1. 주요 테이블 8

4-2. 저장 원칙 9
4-3. 데이터 전달 형식 9
5. API 및 실시간 통신 11
5-1. 주요 REST API 11
5-2. 면접 실시간 통신 (WebSocket · WebRTC) 12
5-3. 동시 면접 입장 관리 12
6. 운영 서버 및 AI 모델 배치 13
6-1. 운영 서버 사양 13
6-2. 모델 배치 기본안 13
6-3. 모델 배치 근거 14
6-4. 대안 배치안 14
7. 면접 진행·질문 생성·이력서 처리 14
7-1. 응시 상태 전이 14
7-2. 장애 판단 15
7-3. 화면 이탈 감지 15
7-4. 질문 생성 절차 16
7-5. 답변 처리 및 꼬리질문 17
7-6. 이력서 내용 추출 및 OCR 17
8. 채점 및 1차 합격 결정 18
8-1. 채점 흐름 18
8-2. 단계별 구현 방식 18
8-3. 실패·지연 처리 19
8-4. 하이라이트 구간 선정 19
9. 코딩테스트 실행 환경 19
10. 결과 공개·알림·자동 작업 19
11. 보안·개인정보 20
12. 성능 목표 및 측정 21
13. 테스트 계획 22
13-1. 완성 기준별 테스트 방법 22
13-2. 운영 서버 측정 계획 22
14. 설정값 목록 23
15. 확인 필요 사항 25
부록. PRD 요구사항 추적표 26

TRD란? PRD에 적은 기능을 "어떻게 만들지" 정하는 문서다. 서버를 어떻게 나눌지, 어떤 프로그램·라이브러리를 쓸지, 데이터를 어떻게 저장할지, 화면과 서버가 무엇을 주고받을지, AI를 어떻게 부를지, 어떻게 테스트할지를 적는다. 사용자 입장의 규칙이 바뀌면 PRD를 먼저 고치고 TRD를 따라 고친다.

1. 개요 및 설계 원칙

1-1. 목적 및 범위

PRD v1.6의 1차 면접 자동화 전체(공고·채점 기준 정하기 → 지원·준비 → AI 면접 → 코딩테스트 → AI 채점·1차 합격 여부 결정 → 결과 공개 → 자동 삭제)를 서버 1대(그래픽카드 메모리 6GB · 일반 메모리 58GB)에서, 외부 서비스 없이 만들기 위한 기술 기준을 정한다. 2차 면접, 외부 시스템 연동, 모바일 앱은 범위 밖이다(PRD 2-2).

1-2. 설계 원칙

원칙	어떻게 만든다
AI가 평가하고, 시스템은 결과의 유효성을 검증한다	AI는 지원자의 답변과 평가 기준을 바탕으로 항목별 점수, 종합 평가 점수, 1차 합격 여부와 그 이유를 판단한다. 서버는 AI 응답의 데이터 형식과 값의 범위, 근거 문장이 실제 답변에 존재하는지를 검증한다. 검증을 통과하지 못한 결과는 확정하지 않고 재시도하거나 '확인 필요' 상태로 처리한다. 합격 여부를 서버가 별도 점수 규칙으로 재판정하지 않는다.
면접 진행 상태와 업무 규칙은 시스템이 관리한다	AI는 질문 생성, 답변 분석, 꼬리질문 판단 및 평가를 수행한다. 면접 시작·종료, 질문 순서, 답변 제한시간, 꼬리질문 횟수, 재접속, 결과 공개 등은 서버가 정해진 규칙에 따라 관리한다. AI가 다음 단계로 이동할지 임의로 결정하지 않으며, 허용된 상태 전이만 수행한다.
면접은 실시간으로 진행하고, 처리 결과는 순서에 맞게 전달한다	지원자의 음성은 WebRTC로 실시간 전송하고, STT 결과는 WebSocket을 통해 자막으로 전달한다. 음성 수신과 STT 처리를 분리해 추론 지연이 면접 연결을 불필요하게 막지 않도록 한다. 질문, 답변, 자막 및 진행 상태는 해당 면접의 순서와 연결 관계를 유지한다.
일부 기능의 실패가 전체 면접의 중단으로 이어지지 않게 한다	질문 생성, STT, TTS, AI 평가 및 백그라운드 작업에는 실패별 처리 방법을 정의한다. 가능한 경우 재시도하거나 질문은행·예비 질문 또는 텍스트 표시로 대체한다. 평가 결과를 안전하게 확정할 수 없는 경우 임의의 결과를 만들지 않고 '확인 필요' 상태로 전환한다.
필요한 데이터만 수집하고 정해진 기간만 보관한다	이력서, 음성, 답변, 코드 및 평가 결과는 서비스 기능에 필요한 범위에서 처리한다. 개인정보 접근은 역할과 데이터 소유권에 따라 서버에서 통제한다. 원본과 파생 데이터는 각각 정해진 보관·삭제 정책을 따르며, 외부 AI 서비스로 개인정보나 면접 데이터를 전송하지 않는다.
기능 설정과 프로그램 로직을 분리한다	동시 면접 수, 재시도 횟수, 시간 제한 및 결과 공개·삭제 관련 설정은 프로그램 코드에 고정하지 않는다. 변경 가능한 설정은 중앙에서 관리하고, 설정 변경은 적용 범위와 시점을 명확히 하며 필요한 감사 기록을 남긴다.
검증 가능한 기준으로 구현 완료 여부를 판단한다	주요 기능에는 요구사항과 연결되는 테스트 기준을 둔다. 면접 상태 전이, 질문 구성, AI 결과 검증, 권한 검사, 장애 복구, 데이터 삭제 및 성능 목표를 테스트로 확인한다. 실제 실행이나 측정으로 확인하지 않은 기능과 성능을 완료된 것으로 간주하지 않는다.

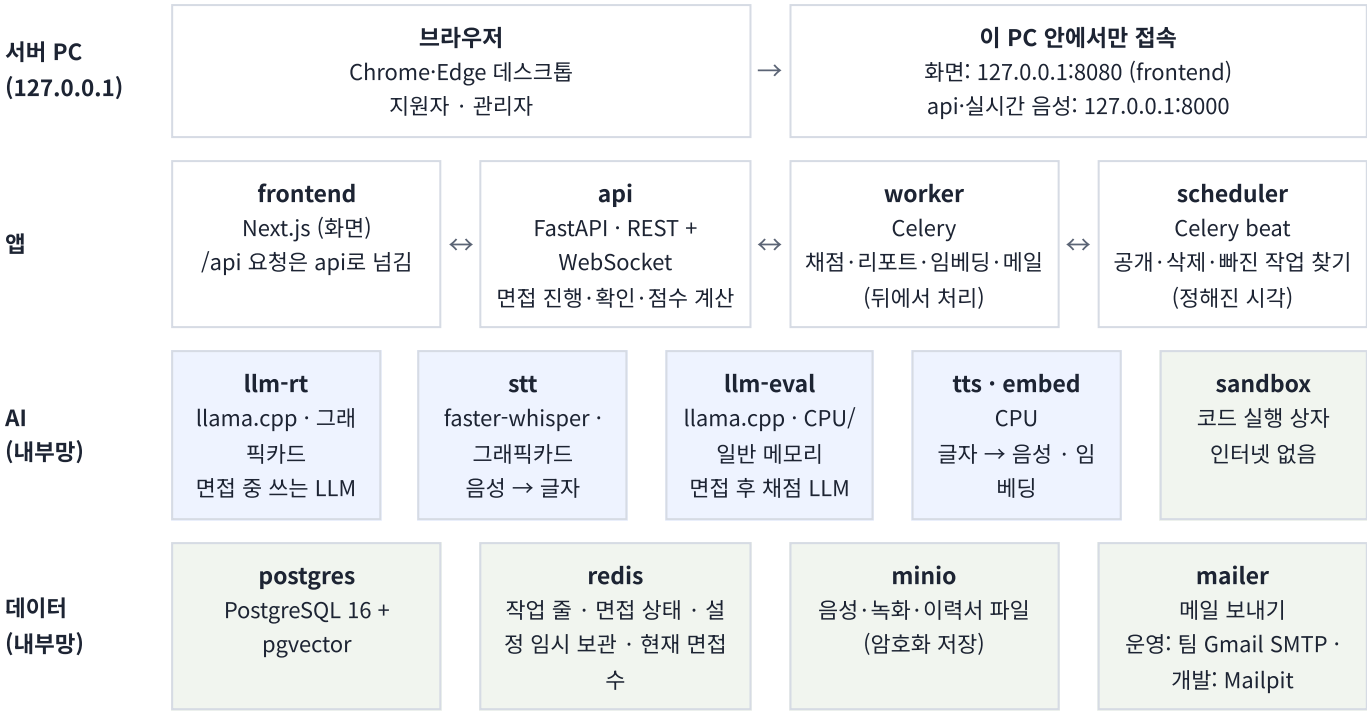
1-3. 범위 제약 (관리자 1명 기준)

1. **계정**: **관리자 계정 1개**로 운영한다. 회사별 계정, 회사마다 데이터를 나누는 구조(멀티테넌시)는 만들지 않는다.
2. **공고 넣기**: 관리자가 직접 쓰거나, 공공 채용정보 Open API나 더미 데이터(연습용 가짜 공고 파일)로 불러온다. 회사명·회사 소개는 공고의 칸으로 둔다.

3. **그대로 지키는 것**: 지원자·관리자 역할 구분, 모든 API에서 역할과 데이터 주인 확인(지원자는 다른 지원자 데이터를 못 봄), 개인정보 동의, 감사 로그(누가 언제 무엇을 했는지), 보관·삭제 규칙.
4. **만들지 않는 것**: 회사 번호(tenant_id) 칸, 회사 구분 공통 처리, 회사별 DB 분리, 회사 담당자 관리, 회사끼리 권한 확인.

2. 시스템 구성

2-1. 시스템 구성도



요청 흐름

1. **실행**: 서버 PC 한 대에서 모든 프로그램을 Docker 컨테이너에 담아 docker compose 파일 하나로 띄운다. 포트는 `127.0.0.1` 에만 연결해(예: `127.0.0.1:8080:3000`) 그 PC 안의 브라우저에서만 접속할 수 있다.
2. **화면과 일반 요청**: 브라우저는 frontend(`http://127.0.0.1:8080`)에 접속한다. 화면에서 보내는 `/api` 요청은 frontend가 Next.js 주소 넘기기(rewrites)로 api에 넘긴다. 주소가 하나라서 로그인 쿠키가 그대로 쓰인다.
3. **면접 중 음성·자막**: 브라우저가 api(`127.0.0.1:8000`)에 바로 붙는다. 답변 음성은 WebRTC로 실시간 전송하고, 질문·자막·진행 상태는 WebSocket(`ws://127.0.0.1:8000`)으로 주고받는다(5-2). api는 연결이 우리 화면 주소에서 왔는지(Origin 헤더) 확인하고, 아니면 끊는다.
4. **HTTPS·문지기 없음**: 이 PC 안에서만 접속하므로 HTTPS 인증서와 문지기 프로그램(NGINX)을 쓰지 않는다. 브라우저는 `127.0.0.1` · `localhost` 주소에서는 HTTPS 없이도 마이크·카메라를 열어 준다. 다른 PC에서 접속하게 바꾸면 HTTPS와 443번 포트 하나로 받는 문지기가 필요하다(11장).
5. **내부망**: AI와 데이터 상자는 서버끼리만 통하는 내부망(`internal: true`)에만 연결한다.
6. **밖으로 나가는 연결**: api는 Open API로 공고를 불러올 때만, mailer는 Gmail 메일 서버(smtp.gmail.com, 587번 포트)로 메일을 보낼 때만 나간다(11장). 개발 중에는 Mailpit(밖으로 보내지 않는 시험용 메일 서버)을 쓴다.

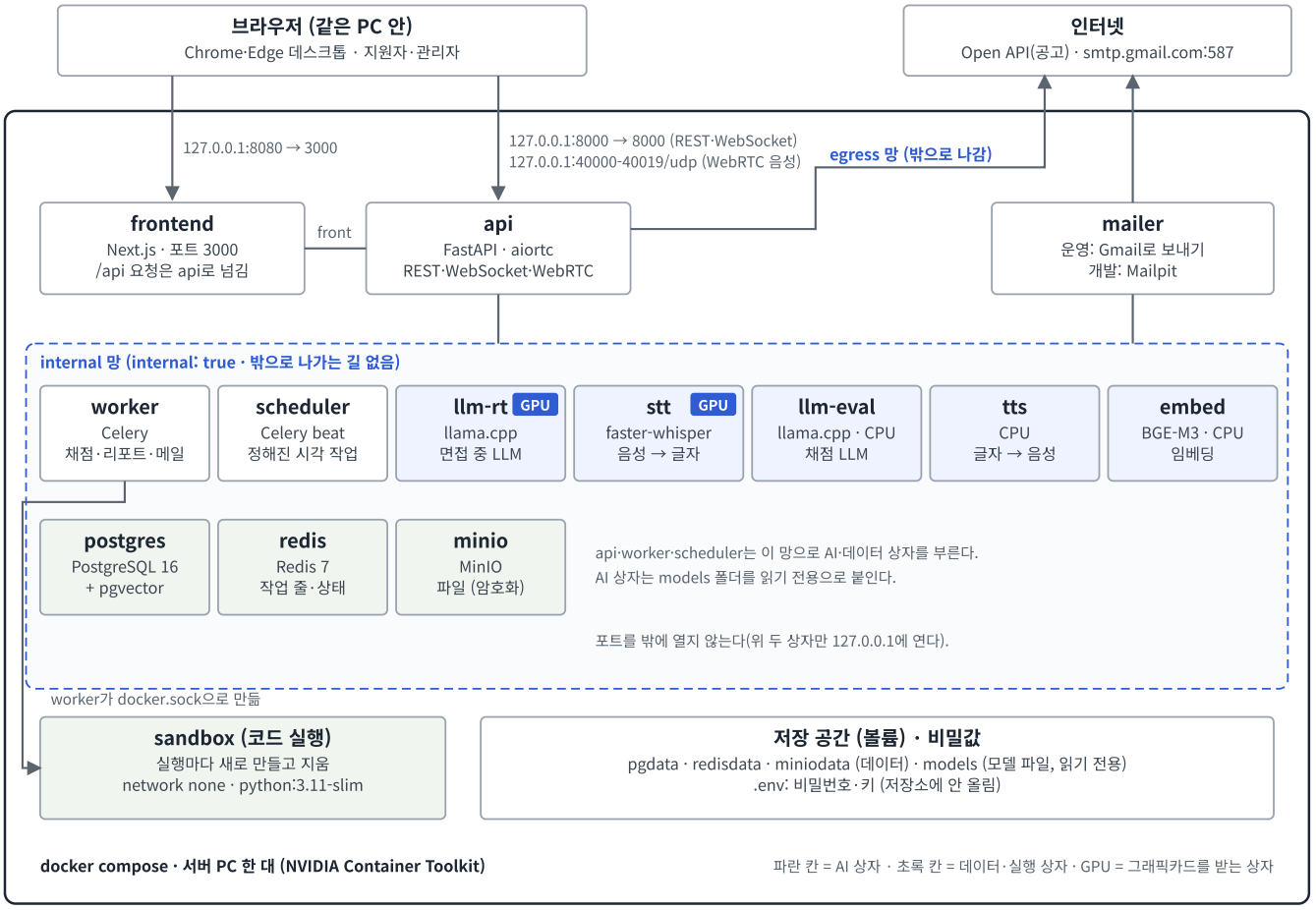
쓰지 않는 것: 쿠버네티스(서버 여러 대 관리 도구), 별도 모니터링 도구, 부하 테스트 도구.

2-2. 서비스별 역할

프로그램	맡는 일

frontend	화면, 장치 점검(마이크·카메라 열기, 소리 크기 재기), 전체 화면·화면 이탈 알아채기(탭 전환·창 포커스 잃음·전체 화면 해제 이벤트), 녹음·녹화를 조각내 올리기, 몇 초마다 "아직 연결돼 있음" 신호(heartbeat) 보내기
api	로그인·권한, 공고·채점 기준 만들기·고치기와 잠금, 응시 상태 관리, 면접 진행 그래프(LangGraph, 7-5) 돌리기, 면접 중 LLM·STT 부르기, 동시 면접 입장 관리, 장애·이탈 판단, 점수 계산, AI 결정 확인, 결과 조회
worker	질문 만들기 그래프(LangGraph, 7-4) 돌리기, 답변마다 미리 채점, 최종 채점·AI 결정, 리포트·피드백 쓰기, 이력서 내용 뽑기(OCR 포함), 임베딩, 코드 채점, 메일 보내기
scheduler	결과 한꺼번에 공개(매일 10:00), 보관 기간 지난 데이터 삭제(매일), 빠지거나 멈춘 작업 다시 넣기(5분마다), 공고 자동 마감, 안내·삭제 예정 알림

2-3. 도커 배포 구성도



도커 실행 방식

1. **한 번에 띄우기**: `docker compose up -d` 하나로 그림의 상자를 모두 띄운다. 상자마다 `restart: unless-stopped`를 둬서 멈추면 다시 켠다.
2. **켜지는 순서**: postgres·redis·minio가 준비됐는지 확인(healthcheck)한 뒤에 api·worker·scheduler를 켠다 (`depends_on`).
3. **여는 포트**: frontend 8080, api 8000, WebRTC 음성용 UDP 40000~40019만 `127.0.0.1`에 연다. 나머지 상자는 포트를 열지 않는다.
4. **망 3개**: front(frontend·api), internal(api·worker·scheduler·mailer와 AI·데이터 상자, `internal: true`), egress(api·mailer)로 나눈다. 밖으로 나갈 수 있는 상자는 api와 mailer뿐이다.
5. **WebRTC 음성**: aiortc가 쓰는 UDP 포트를 위 범위로 묶고, 브라우저에 알리는 접속 주소를 `127.0.0.1`로 바꾼다. 상자 안 주소를 알리면 브라우저가 붙지 못한다.

6. **그래픽카드**: llm-rt와 stt에만 NVIDIA 그래픽카드를 붙인다(`gpus` 설정). 서버 PC에 NVIDIA Container Toolkit을 설치해 둔다.
7. **저장 공간**: DB·Redis·MinIO 데이터는 이름 붙인 볼륨(pgdata·redisdata·miniodata)에 둬서 상자를 지웠다 다시 만들어도 남는다. 모델 파일은 models 폴더 하나에 두고 AI 상자에 읽기 전용으로 붙인다.
8. **sandbox**: 늘 떠 있는 상자가 아니다. worker가 Docker 소켓(`docker.sock`)으로 코드를 실행할 때마다 새로 만들고 끝나면 지운다(9장). Docker 소켓은 worker에만 붙인다.
9. **비밀값**: Gmail 앱 비밀번호, 암호화 키, DB 비밀번호는 `.env` 에서만 읽고 저장소에 올리지 않는다. 개발용 값은 `.env.dev` 에 둔다.

3. 기술 스택 및 버전 (Tech Stack)

구성요소	선택	버전 기준	선정 근거	비고
언어	Python	**3.11.x**	LangChain·LangGraph 1.x가 3.10 이상을 요구하고, PaddleOCR·CTranslate2·PyTorch가 3.11용 설치 파일을 모두 낸다	api·worker·코드 실행 상자를 모두 같은 버전으로 맞춘다
화면	Next.js (App Router) · React · TypeScript	**Next.js 16.x · React 19.x · TypeScript 5.x**	Next.js 16이 현재 안정판이고 15는 유지보수만 받는다. App Router는 React 19가 필요하다	지원자·관리자 화면. 보안 패치가 잦아 16.x 최신 패치로 올린다
코드 편집기	Monaco Editor (@monaco-editor/react)	**4.x**	React 19에서 쓸 수 있는 Monaco용 React 연결 패키지다	코딩테스트 답안 입력 칸
서버	FastAPI · Uvicorn	**FastAPI 0.115 이상**	Pydantic v2만 쓰는 버전이고 WebSocket을 기본으로 지원한다	REST API, 면접 중 WebSocket
데이터 형식 확인	Pydantic	**2.x**	FastAPI와 LangChain 1.x가 v2를 쓴다. v1과 섞으면 형식 검사가 두 갈래로 나뉜다	요청·응답·LLM 답 검사(4-3)
DB 다루기	SQLAlchemy · Alembic · psycopg	**SQLAlchemy 2.0.x · Alembic 1.13 이상 · psycopg 3.x**	2.0의 비동기 세션을 FastAPI와 함께 쓴다. psycopg 3는 LangGraph의 PostgreSQL 체크포인트가 쓰는 드라이버다	테이블 구조 변경은 Alembic으로만 한다
뒤에서 하는 작업	Celery · Celery beat	**5.4 이상**	Python 3.11과 Redis 7을 지원하는 안정판이고, 정해진 시각 작업(beat)이 같은 패키지에 들어 있다	채점·질문 만들기·OCR·메일, 결과 공개·삭제(10장)
임시 저장소	Redis	**7.x**	작업 줄, 음성 조각 스트림(Streams), 입장 관리용 Lua 스크립트(5-3)를 모두 지원한다	이미지 `redis:7`, 데이터는 볼륨에 저장
DB	PostgreSQL · pgvector	**PostgreSQL 16.x · pgvector 0.8 이상**	16은 2028년까지 보안 패치를 받는다. pgvector 0.8부터 조건(질문 종류)을 건 유사도 검색에서도 결과 수가 모자라지 않는다	이미지 `pgvector/pgvector:pg16`. LangGraph 진행 상태도 저장
파일 저장	MinIO	**마지막 무료판 이미지 태그로 고정**	저장할 때 암호화(4-2)와 5분짜리 임시 재생 주소(8-4)를 기본 기능으로 쓴다	무료판은 2025년 10월부터 새 이미지가 나오지 않는다. 보안 패치가 없으므로 내부망에만 두고 포트를 열지 않는다

LLM 띄우기	llama.cpp server	**빌드 번호 고정 (CUDA 이미지)**	버전 번호 없이 매일 새 빌드가 나온다. Qwen3를 읽을 수 있는 빌드 중 13-2 측정에 쓴 빌드로 묶는다	이미지 `ghcr.io/ggml-org/llama.cpp:server-cuda` (6장)
LLM	Qwen3 (GGUF)	**모델 크기·양자화 (Q4_K_M) 고정**	한국어를 지원하고, 생각 과정 출력을 끌 수 있으며(`/no_think`), 6GB 그래픽카드에 맞는 크기가 있다	모델 파일의 해시(sha256)를 적어 두고 같은 파일만 쓴다
LLM 부르기	LangChain · langchain-openai	**1.x**	1.0부터 큰 변경 없이 유지하는 안정판이다. JSON 형식 강제(`with_structured_output`)와 `ChatOpenAI` 를 포함한다	모든 LLM 요청
대화 흐름 제어	LangGraph · langgraph-checkpoint-postgres	**1.x**	1.0 안정판이고, 멈춰 기다리기(interrupt)와 PostgreSQL 체크포인트를 공식 지원한다	면접 진행(7-5)·질문 만들기(7-4)
실시간 음성 전송	aiortc (WebRTC)	**1.9 이상**	파이썬에서 WebRTC 음성을 받을 수 있고, asyncio로 돌아 FastAPI와 같은 흐름에서 쓸 수 있다	UDP 포트 범위를 고정해야 한다 (2-3)
음성 → 글자 (STT)	faster-whisper · Whisper Large V3 Turbo	**faster-whisper 1.1 이상**	Large V3 Turbo 모델을 바로 불러올 수 있는 버전부터다. CUDA 12·cuDNN 9용 CTranslate2 4.x를 함께 쓴다	한국어 고정, int8_float16
글자 → 음성 (TTS)	설치형 한국어 TTS (후보: Supertonic)	**한국어 품질 확인 뒤 고정**	CPU에서 짧은 문장을 빠르게 만들 수 있다	만들지 못하면 자막만 쓴다
임베딩	BGE-M3 · sentence-transformers	**sentence-transformers 3.x 이상**	BGE-M3를 바로 불러와 CPU로 돌릴 수 있다	1024차원 숫자 목록. 비슷한 질문 찾기·RAG(7-4)
PDF 글자 꺼내기	pdfplumber · pypdfium2	**pdfplumber 0.11 이상 · pypdfium2 4.x**	글자와 위치를 함께 꺼낸다. pypdfium2는 다른 프로그램을 깔지 않고 쪽을 이미지로 바꾼다	스캔한 쪽만 OCR로 넘긴다(7-6)
OCR	PaddleOCR	**3.x (PP-OCRv5 한국어 모델)**	3.x부터 인식 정확도를 높인 새 모델(PP-OCRv5)을 쓰고 CPU로 돈다	대안: Tesseract 5 (kor)
코드 실행 상자	Docker · python:3.11-slim	**3.11**	서버와 같은 파이썬 버전이라 실행 결과가 어긋나지 않는다	인터넷 없음(9장)
메일	Gmail SMTP · Mailpit	**Mailpit 1.x**	운영은 팀 Gmail, 개발은 밖으로 보내지 않는 시험용 메일 서버를 쓴다	앱 비밀번호는 `.env` 에만
실행	Docker Engine · Docker Compose · NVIDIA Container Toolkit	**Compose v2 (`docker compose`)**	compose 파일 하나로 모든 상자와 그래픽카드 연결을 정할 수 있다	127.0.0.1에서만 실행(2-3)

버전 기준은 2026년 10월 기준이다. 정확한 버전은 처음 구현할 때 고정 파일로 묶는다.

구현 준수 사항

1. **LLM 연결**: LangChain `ChatOpenAI` 에 내부망 주소(예: `http://llm-rt:8080/v1`)를 넣는다. llama.cpp가 OpenAI와 같은 요청 형식을 쓰기 때문이며, OpenAI 회사 서버로는 나가지 않는다.
2. **LLM 답 형식**: `with_structured_output` 으로 JSON 스키마를 강제하고, 받은 답은 Pydantic으로 다시 검사한다.
3. **Qwen3**: 생각 과정 출력을 끈다(`/no_think`). 면접 중용·채점용 구성은 6장.

4. **STT**: 한국어 고정, 메모리를 줄인 계산 방식(int8_float16).
5. **Gmail**: 2단계 인증 뒤 받은 앱 비밀번호를 서버 `.env`에만 두고 저장소에 올리지 않는다. 하루 보낼 수 있는 양(약 500통)은 설정값 `mail_daily_limit`(14장).
6. **버전 고정**: 위 버전 기준 안에서 처음 구현할 때 정확한 버전을 고정 파일(requirements.lock, package-lock.json)로 묶는다. 도커 이미지도 태그로 고정하고 `latest`는 쓰지 않는다.

4. 데이터 설계 (Data Structure)

4-1. 주요 테이블

테이블마다 어떤 정보를 담고 무슨 일을 하는지만 적는다. 칸 하나하나의 이름과 뜻은 별도 문서인 테이블 정의서에 둔다.

테이블	담는 정보와 하는 일
users (회원)	지원자와 관리자의 계정이다(관리자는 1명). 로그인 정보, 이메일 인증 여부, 로그인 실패에 따른 잠금, 탈퇴 여부를 담는다.
job_postings (공고)	채용 공고의 내용, 접수 기간, 게시 상태, 결과 공개 시각을 담는다. 관리자가 직접 쓴 공고인지, Open API나 더미 데이터로 불러온 공고인지도 기록해 같은 공고를 두 번 넣지 않게 한다. 직무 종류는 job_codes 테이블로 따로 관리한다.
evaluation_criteria (채점 기준)	공고마다 채점 기준을 담는다. 항목 5개의 비율, 코딩테스트를 쓸지와 반영 비율, 합격 기준 점수다. 버전을 붙여 두고, 잠근 뒤에는 바꾸지 못하게 한다.
applications (지원 1건)	지원 1건에 관한 상태를 한곳에 모으는 중심 테이블이다. 이력서 사본과 원본 파일 위치, 이력서 글자를 어떻게 읽었는지(OCR 포함), 응시·채점·결과·공개 상태, 지금 동시 면접 자리를 잡고 있는지(5-3), 결과 공개 시각과 삭제 예정 시각을 담는다.
consents (동의)	지원할 때 받은 개인정보 동의를 담는다. 어떤 문구 버전에 무엇을 동의했는지, 나중에 철회했는지 남긴다.
questions (질문 DB)	면접 질문을 모아 두는 곳이다. 엑셀에서 불러온 질문, AI가 만든 질문, 예비 질문이 모두 들어간다. 비슷한 질문을 찾는 데 쓰는 임베딩과 쓰인 횟수도 둔다(7-4).
interview_questions (이 면접에 낸 질문)	지원자 한 명의 면접에 실제로 낸 질문 목록이다. 낸 순서와 그때의 질문 사본, 꼬리질문인지, 다시 만든 횟수, 대신 쓴 질문인지를 남긴다.
answers (답변)	질문마다 지원자의 답변을 담는다. 음성 파일 위치, 글자로 바꾼 답변, 답변 시작·끝 시각이다.
item_scores (항목 점수)	답변마다 AI가 매긴 항목 점수(0~5)와 근거 문장, 그 근거가 실제 답변에 있는지 확인한 결과를 담는다(8장).
coding_submissions (코드 제출)	코딩테스트 문제마다 제출한 코드와 자동 저장 기록, 테스트 통과 수, 붙여넣기 시도 횟수를 담는다(9장).
results (결과)	지원 1건의 최종 결과다. 점수 합계, AI가 정한 합격 여부와 이유, 시스템이 확인한 결과, 그때 쓴 모델·지시문 버전, 지원자에게 보여 줄 내용을 담는다(8장).
session_events (면접 중 기록)	면접 중에 일어난 일(연결 끊김, 창 닫기, 화면 이탈 등)과 그것을 장애로 봤는지 부정행위로 봤는지를 남긴다(7-2, 7-3).
highlights (하이라이트 구간)	관리자가 다시 들어 볼 답변 구간(잘한 곳, 점수가 깎인 곳)을 담는다(8-4).

LangGraph 체크포인트 테이블	면접 진행 그래프와 질문 만들기 그래프의 진행 상태를 담는다. LangGraph가 직접 만들고 관리하며, 끊긴 면접을 이어 갈 때 쓴다(7-5).
memos · notifications · audit_logs · deletion_logs	관리자 메모, 보낼 알림 목록, 누가 언제 무엇을 했는지 남기는 감사 로그, 지운 데이터 기록이다.
app_settings (설정)	서버를 다시 켜지 않고 바꿀 수 있는 설정값을 담는다. 목록은 14장.

4-2. 저장 원칙

- 질문 사본, 이력서 사본, 채점 기준 버전은 응시 때의 값을 복사해 두고, 나중에 원본이 바뀌어도 바꾸지 않는다(QST-03, APP-05, JOB-07).
- 개인정보 칸(phone, 이력서 사본)과 MinIO에 저장한 파일은 AES-256으로 암호화한다(11장).
- 보관 기간이 끝나 지울 때는 DB 행 삭제 + MinIO 파일 삭제 + 거기서 만든 데이터(임베딩·임시 보관 데이터·Redis 키) 삭제를 한 번의 작업으로 하고, deletion_logs에 남긴다.

4-3. 데이터 전달 형식

백엔드가 데이터를 받고 넘기는 모양을 예시로 정한다. 실제 칸 목록은 테이블 정의서를 따른다.

공통 규칙

1. **형식 정의**: 모든 요청·응답·메시지·LLM 답은 Pydantic 모델로 정의하고 JSON으로 주고받는다. 모델에 없는 칸이 오면 거절한다(`extra="forbid"`).
2. **값 규칙**: id는 UUID 문자열, 시각은 한국 시간이 붙은 ISO 8601(`2026-11-03T10:00:00+09:00`), 점수는 소수 오차를 막기 위해 문자열(`"70.15"`)로 보낸다.
3. **오류 응답**: 모든 오류는 `{"code", "message"}` 한 가지 모양으로 보낸다. 화면은 code로 안내 문구를 고른다.
4. **개인정보**: 이름·연락처는 화면 응답에만 넣고, LLM 입력과 Redis 메시지에는 넣지 않는다.

① 면접 시작 (화면 → api, REST)

```
POST /applications/{application_id}/interview/start

# 응답 200: 입장 성공
{
    "session_id": "5d1c2f0e-…",                  # 면접용 출입증 번호 (sid)
    "ws_url": "ws://127.0.0.1:8000/ws/interview/5d1c2f0e-…",
    "question_total": 10                         # 기본 질문 수
}

# 응답 429: 자리 꽉 참 (응시 횟수 안 씀)
{ "code": "SEATS_FULL", "message": "잠시 뒤 다시 시도해 주세요." }
```

② 질문 보내기 (api → 화면, WebSocket)

```
{
    "type": "question",
    "seq": 3,                                    # 질문 번호
    "kind": "base",                              # base 기본 / follow_up 꼬리
    "text": "최근 프로젝트에서 모델 성능을 높인 방법을 설명해 주세요.",
    "audio_url": "/media/questions/5d1c…/3.wav",   # 음성을 못 만들었으면 null
    "answer_sec": 90,                            # 남은 답변 시간
    "progress": "3 / 10"
}
```

③ 실시간 자막 (api → 화면, WebSocket)

```
{
    "type": "caption",
    "seq": 3,                                  # 질문 번호
    "answer_no": 1,                            # 같은 질문 안의 답변 번호
    "chunk_no": 7,                             # 화면은 이보다 작은 번호가 늦게 오면 버린다
    "text": "저는 이전 프로젝트에서 데이터 불균형을",
    "is_final": false                          # true면 채점에 쓰는 최종 글자
}
```

④ 음성 조각 (api → Redis 스트림 → stt, 내부)

```
# 스트림 이름: audio:{session_id}   조각 하나 = 1초
{
    "seq": 3,
    "answer_no": 1,
    "chunk_no": 7,
    "pcm": <16kHz 모노 음성 바이트>,
    "received_at": "2026-11-03T10:04:12+09:00"
}
```

⑤ 항목 채점 (worker ↔ 채점 LLM)

```
class ItemScore(BaseModel):
    item: Literal["expertise", "logic", "relevance", "delivery", "culture_fit"]
    score: int = Field(ge=0, le=5)             # 0~5 정수
    quote: str                                 # 답변 원문에 그대로 있어야 함

class ScoreResult(BaseModel):
    items: list[ItemScore]

# LLM이 돌려준 답 (with_structured_output으로 형식 강제)
{ "items": [
    { "item": "expertise", "score": 4, "quote": "클래스 가중치를 조정해 재현율을 높였습니다" },
    { "item": "logic",     "score": 3, "quote": "먼저 원인을 확인하고 그다음 방법을 골랐습니다" }
] }
```

⑥ 합격 여부 결정 (worker ↔ 채점 LLM, results에 저장)

```
# LLM에 넣는 값: 서버가 계산한 점수 + 관리자 기준
{ "item_scores": { "expertise": "80.00", "logic": "65.00", … },
  "interview_score": "72.50", "coding_score": "60.00", "total_score": "68.75",
  "pass_score": "70.00" }

# LLM이 돌려준 결정 (서버는 형식·값 범위·근거만 검증)
{ "decision": "fail",
  "reason": "직무 이해는 좋지만 구체적인 근거가 부족했다. …",
  "quotes": ["먼저 원인을 확인하고 그다음 방법을 골랐습니다"] }
```

⑦ 지원자 결과 (api → 화면, REST)

```
GET /me/results/{application_id}              # 공개 시각 전에는 404

{
    "application_id": "7b2e9c41-…",
    "result": "fail",                              # pass / fail
    "strengths": ["직무 개념을 정확히 설명함"],
    "improvements": ["경험을 수치와 함께 말하면 좋음"],
    "status": "released"                           # 점수·근거·이유는 넣지 않음
}
```

5. API 및 실시간 통신

5-1. 주요 REST API

요청은 "방식 + 주소"로 적는다. 방식은 GET 읽기, POST 만들기·실행, PATCH 일부 고치기, PUT 통째로 바꾸기다. 본문의 숫자는 서버의 대답 번호로, 401은 로그인 필요, 403은 권한 없음, 404는 없음(또는 보여 줄 수 없음), 409는 지금 상태에서는 안 됨, 429는 자리가 꽉 참을 뜻한다.

방식 · 주소	하는 일
POST /auth/signup · /auth/login · /auth/password-reset	가입·로그인·비밀번호 재설정. 이메일 인증은 설정 `email_verification_required`가 true일 때만 지원 API에서 확인한다
GET /postings · /postings/{id}	공개 공고 목록·상세 (로그인 필요 없음)
POST /postings · PATCH /postings/{id} · POST /postings/{id}/publish · /close	관리자 공고 관리(`/admin` 아래, 관리자 확인 `require_admin`). 기준이 잠긴 뒤 기준 칸을 바꾸려 하면 409
POST /admin/postings/import	Open API나 더미 JSON/CSV 파일에서 공고를 불러온다. external_id로 같은 공고 두 번 넣기를 막고, 임시저장(draft) 상태로 저장한다. Open API 호출은 api 상자에서만 밖으로 나가고, 공고 정보만 가져온다
POST /postings/{id}/draft-ai	AI 공고 초안 만들기(채점용 LLM, 뒤에서 처리)
POST /applications · /applications/{id}/consent · /resume	지원·동의·이력서 내기 (내용 뽑기는 worker가 함)
POST /applications/{id}/precheck	인터넷·AI 서버 상태 점검 결과 돌려주기
POST /applications/{id}/interview/start	입장 관리(5-3)를 통과하면 면접용 출입증(세션 토큰)을 준다. 꽉 찼으면 429와 안내 코드, 응시 횟수는 쓰지 않음
(WebRTC) 답변 음성	파일로 올리지 않고 WebRTC로 실시간 전송한다. 연결 준비는 WebSocket으로 한다(5-2)
POST /coding/{sid}/run · /autosave · /submit	공개 예시로 실행, 30초마다 자동 저장, 최종 제출
GET /me/results/{application_id}	공개 시각 이후에만 지원자용 결과(public_view)를 준다. 그 전에는 404
GET /admin/postings/{id}/applicants · /applicants/{aid} · /compare	관리자 결과 보기(읽기만). 결정을 고치거나 승인하는 주소는 없다
POST /admin/applicants/{aid}/memo	내부 메모
PUT /admin/settings/{key}	관리자 설정 바꾸기(N 등). 감사 로그에 남김

POST /admin/analysis/{aid}/reprocess	'확인 필요' 건 다시 돌리기

5-2. 면접 실시간 통신 (WebSocket · WebRTC)

면접 중에는 연결 두 개를 계속 열어 둔다. 답변 음성은 WebRTC로, 나머지(질문, 자막, 진행 상태, 연결 확인)는 WebSocket으로 주고받는다.

답변 음성·자막 처리 흐름

1. **연결 준비**: 면접 시작 때 화면과 api가 WebSocket으로 WebRTC 연결 정보(`rtc_offer`·`rtc_answer`·`rtc_ice`)를 주고받는다. 같은 PC(127.0.0.1) 안이라 외부 중계 서버(STUN·TURN)는 쓰지 않는다.
2. **받기**: api는 aiortc로 음성을 받아, 답변 번호를 붙인 1초 단위 조각으로 Redis 스트림(순서가 지켜지는 작업 줄)에 넣고 MinIO에 답변 음성 파일로 이어 쓴다. api는 STT를 기다리지 않는다.
3. **실시간 자막**: stt는 스트림에서 조각을 꺼내 `caption_interval_sec`마다 지금까지의 음성을 글자로 바꿔 임시 자막을 만든다. 그래픽카드가 바쁘면 임시 자막은 건너뛴다. 면접 진행과 최종 글자에는 영향이 없다.
4. **보내기**: api가 자막을 WebSocket `caption` 메시지로 화면에 보낸다. 메시지마다 질문 번호·답변 번호·조각 번호가 있어, 화면은 순서가 뒤바뀌거나 지난 질문의 자막이면 버린다.
5. **최종 글자**: 답변이 끝나면 stt가 그 답변의 전체 음성을 한 번 더 글자로 바꿔 최종 글자(`is_final`)를 만든다. 꼬리질문 판단과 채점은 이 최종 글자만 쓴다(7-5).

방향	메시지 이름	내용
화면 ↔ 서버	rtc_offer · rtc_answer · rtc_ice	WebRTC 연결 준비 정보
서버 → 화면	caption	질문 번호, 답변 번호, 조각 번호, 자막 글, 최종 여부(is_final)
서버 → 화면	question	질문 번호, 종류(기본/꼬리), 자막 글, 질문 음성 주소(없으면 null), 답변 시간, 진행 표시('3 / 10')
화면 → 서버	heartbeat	5초마다 "연결돼 있음" 신호. 서버는 3번 연속(15초) 안 오면 장애 후보로 기록
화면 → 서버	answer_done · unload · visibility · blur · fullscreen · device_error	답변 완료, 지원자가 창을 닫음/새로고침(beforeunload·pagehide 이벤트), 화면 이탈 신호, 장치 오류
서버 → 화면	warn · terminate · paused · resume	1번째 이탈 경고(멈춘 시점의 남은 답변 시간 포함), 2번째 강제 퇴장(또는 마감 종료), 장애로 멈춤 안내, 이어가기 다시 시작(이탈 뒤에는 남은 시간부터)

5-3. 동시 면접 입장 관리

- Redis에 "지금 진행 중인 면접 수"(`interview:active`)를 두고, 시작 요청이 오면 설정값 N과 비교해 자리가 있을 때만 1을 더한다. 두 사람이 동시에 눌러도 N을 넘지 않게 비교와 더하기를 끼어들 틈 없이 한 번에 처리한다(Redis 안에서 한 번에 실행되는 작은 스크립트, Lua). 면접이 끝나거나 취소되면 1을 뺀다.
- **끊긴 면접의 자리**: 장애로 '장애 중단'이 된 면접은 끊긴 뒤 `disconnect_seat_hold_min`(10분) 동안만 자리를 맡아 두고, 그 시간이 지나면 1을 빼서 다른 사람이 쓰게 한다. 창 닫기·권한 끄기로 끊긴 면접은 끊긴 순간 바로 '응시 종료'가 되므로 그때 1을 뺀다(7-2). 다시 접속(이어가기)할 때는 자리를 아직 맡아 두었든 이미 돌려주었든 N을 다시 확인하지 않고 들어오게 한다(장애를 겪은 지원자에게 유리하게). 자리를 돌려준 뒤 다시 들어온 면접은 숫자에 다시 1을 더하므로 잠깐 N을 넘을 수 있고, 그동안은 아래 자동 보호가 새 입장을 막아 준다. 이어가기 기한은 지금처럼 공고 마감 시각까지다(10분은 자리를 맡아 두는 시간일 뿐 이어가기 기한이 아니다).
- **자리 표시와 다시 맞추기**: 면접마다 "지금 자리를 잡고 있는지"(applications.holds_seat)를 DB에 표시하고, 1을 더하거나 뺄 때 이 표시도 함께 바꿔서 같은 면접이 자리를 두 번 빼거나 두 번 더하지 않게 한다. 서버가 갑자기 꺼지거나 1을 빼는 처리

가 빠지면 Redis 숫자가 실제와 달라질 수 있으므로, `seat_reconcile_interval_min`(5분)마다 holds_seat가 켜진 면접 수를 세어 Redis 숫자를 그 값으로 다시 맞춘다(10장 빠진 작업 찾기와 같이 돈다).

- 추천안 **N에 세는 상태**: '면접 진행 중'과, 면접 중에 생긴 '장애 중단'만 센다. AI 면접 마지막 질문이 끝나 '코딩테스트 진행 중'으로 넘어가면 자리를 돌려준다(코딩테스트는 그래픽카드를 쓰지 않음). 코딩테스트 중 장애 후 다시 접속할 때도 N과 상관없다.
- 추천안 **꽉 찼을 때**: 시작 API가 429와 안내 코드 `CAPACITY_FULL`을 돌려주고, 화면은 안내와 '다시 시도' 버튼을 보여 준다. 서버에 대기 줄이나 자동 입장은 없다. '다시 시도'는 같은 시작 API를 다시 부르며, 연달아 누르지 못하게 버튼을 10초 동안 막는다.
- N은 `PUT /admin/settings/max_concurrent_interviews`로 바꾸고, 다음 시작 요청부터 바로 적용된다(서버 재시작 필요 없음). N을 줄여도 이미 진행 중인 면접은 끊지 않는다.
- 자동 보호: 30초마다 그래픽카드 메모리 사용률(그래픽카드 상태를 읽는 도구 nvidia-smi/NVML로 확인)과 최근 5분 동안의 '답변 끝 → 다음 질문' 시간(100번 중 95번 기준)을 본다. 메모리 92% 이상이거나 다음 질문 목표 시간(`next_question_target_sec`)을 넘는 상태가 계속되면 새 입장만 잠시 막고, 괜찮아지면 다시 연다. 실제 상한 = N과 자동 보호 상태 중 더 작은 쪽.
- N의 처음 값은 13-2에서 다음 질문 목표 시간을 지키는 가장 큰 동시 면접 수로 정한다.

6. 운영 서버 및 AI 모델 배치

6-1. 운영 서버 사양

항목	값	설계에 주는 영향
그래픽카드 메모리	6GB (고정)	면접 중에 쓰는 모델만 그래픽카드에 올린다. 그래픽카드 메모리가 넘쳐 일반 메모리를 빌려 쓰면 크게 느려지므로, 그래픽카드에 올리는 총량을 약 5.4GB 이하로 관리한다
일반 메모리 (RAM)	58GB	면접 후 채점 LLM(CPU로 계산), 임베딩, TTS, DB·Redis·MinIO를 올릴 여유가 있다
대수	서버 1대	모든 상자를 이 서버 하나에서 docker compose로 띄운다. 개발은 팀원 각자 컴퓨터, 최종 테스트는 이 서버

6-2. 모델 배치 기본안

용도	모델 · 형식	어디에 올리나	메모리(추정)	언제 부르나
면접 중 LLM: 질문을 회사에 맞게 다듬기·범위 줄이기·이력서 기반 질문·꼬리질문 판단	Qwen3-4B Instruct, GGUF Q4_K_M	그래픽카드에 전부	모델 약 2.5GB + 대화 기억(동시 요청 1개당 약 0.3GB)	면접 진행 중
음성 → 글자 (STT)	Whisper Large V3 Turbo, int8_float16	그래픽카드	약 1.0GB	답변 중 임시 자막(`caption_interval_sec`마다), 답변 직후 최종 글자
채점 LLM: 채점·AI 합격 여부 결정·리포트·이력서 정리·공고 초안	Qwen3-8B Instruct, GGUF Q4_K_M	CPU·일반 메모리 (그래픽카드가 남으면 일부만 올림)	일반 메모리 약 6~8GB	이력서 정리(먼저), 질문 번호마다 미리 채점(뒤에서), 응시 끝난 뒤
임베딩	BGE-M3	CPU	일반 메모리 약 2.5GB	질문 DB 불러올 때, 질문 고를 때, 질문 만들 때 예시 찾기

OCR	PaddleOCR 한국어 모델	CPU	일반 메모리 약 1~2GB	이력서 올릴 때(면접 전, 뒤에서)
글자 → 음성 (TTS)	한국어 TTS (후보 Supertonic)	CPU	일반 메모리 1GB 미만	기본 질문: 이력서 확정 뒤 미리 만들어 둠. 꼬리질문: 면접 중 판단 직후 바로 만듦(늦으면 자막만, 7-5)

1. **그래픽카드 합계 목표**: 면접 중 LLM 2.5GB + 대화 기억((N + 질문 만들기 자리 수)개) + STT 1.0GB + 실행 환경 약 0.5GB ≤ 5.4GB.
2. **질문 만들기 자리도 센다**: 질문 만들기(7-4 ⑥, 처음 값 1개)도 같은 LLM의 대화 기억을 쓰기 때문이다.
3. **N**: 이 계산으로는 2~3 정도다. 실제 값은 13-2에서 재서 정한다. 숫자는 모델 파일 크기로 짐작한 값이다.
4. **표의 말뜻**: GGUF는 AI 모델 파일 형식이다. Q4_K_M·int8_float16은 정확도를 조금 내주고 메모리를 크게 아끼도록 모델을 줄인(양자화) 버전이다. 대화 기억(KV 캐시)은 LLM이 요청을 처리하는 동안 쓰는 메모리로, 동시에 처리하는 요청이 늘수록 커진다.

6-3. 모델 배치 근거

- 원래 PRD v1의 기준(Qwen3-8B를 그래픽카드 24GB급에서 돌림)을 6GB 장비에 그대로 하면, 8B 모델(약 5GB)만으로 그래픽카드가 거의 차서 STT와 대화 기억을 함께 올릴 수 없다.
- 면접 중 응답 속도(답변이 끝나고 다음 질문이 뜨기까지의 목표 시간, 7-5)는 그래픽카드에서만 지킬 수 있으므로, 면접 중에는 더 작은 4B 모델을 그래픽카드에 둔다.
- 채점과 합격 여부 결정은 응시가 끝나고 10분 안에만 끝나면 되고, 질문 번호마다 미리 채점하므로(EVL-07), 더 똑똑한 8B 모델을 58GB 일반 메모리에서 CPU로 돌린다.

6-4. 대안 배치안

방법	내용	이럴 때 고른다
A. 기본안	면접 중 4B(그래픽카드) + 채점 8B(CPU)	8B CPU 채점이 응시 1건에 10분 안에 끝나고, 4B 꼬리질문 품질이 괜찮을 때
B. 4B 하나만	면접 중·채점 모두 4B(그래픽카드)	8B CPU 채점이 너무 느릴 때. 정답 세트로 채점 일치도(PRD G2)를 다시 확인
C. 8B 하나만	8B를 그래픽카드에, STT는 CPU로	4B 품질이 부족하고 동시 면접 1~2개로 충분할 때. STT가 느려지는 것은 감수

7. 면접 진행·질문 생성·이력서 처리

7-1. 응시 상태 전이

```
[마감 시각] 준비 중 ─▶ 미응시(마감)  /  진행 중·장애 중단 ─▶ 응시 종료(마감, 답한 질문까지 채점)
준비 중 ─시작(입장 관리 통과)─▶ 면접 진행 중 ─마지막 질문─▶ 코딩테스트 진행 중 ─제출/시간 끝─▶ 응시 완료
   │ 429(N 꽉 참): 상태 그대로, 횟수 안 씀            │                          │
   │                                          ├─15초 끊김·장애─▶ 장애 중단 ─다시 접속·점검─▶ (원래 상태)
   │                                          │     (끊긴 뒤 10분이 지나면 자리만 돌려줌, 이어가기는 마감까지)
   │                                          ├─15초 끊김·창 닫기/권한 끄기 기록─▶ 응시 종료(자리 바로 돌려줌)
   │                                          └─화면 이탈 2번─▶ 응시 취소(1차 불합격)
응시 횟수를 쓰는 시점: 첫 질문이 나올 때(attempt_consumed_at)
```

'대기'는 저장하는 상태가 아니다. 자리가 꽉 찼을 때 보여 주는 안내 화면('다시 시도' 버튼)일 뿐이고, 서버의 응시 상태는 '준비 중' 그대로다(5-3).

7-2. 장애 판단

- 서버는 면접마다 화면이 마지막으로 보낸 연결 확인 신호(heartbeat) 시각을 Redis에 두고, 15초가 넘으면 **그 순간 바로** 아래 기준으로 판단한다. 다시 접속할 때까지 판단을 미루지 않는다.
- '장애'로 판단하면 상태를 '장애 중단'으로 바꾸고 시간을 멈춘다(남은 시간 저장). 자리는 `disconnect_seat_hold_min` (10분) 동안만 맡아 두고, 그 뒤에는 돌려준다(5-3). '장애 아님'으로 판단하면 '장애 중단'을 거치지 않고 바로 '응시 종료'로 끝내고 자리를 돌려준다.
- 판단은 끊기기 직전 기록으로 한다. 연결이 끊기기 전 30초 안에 지원자가 창을 닫은 기록(`unload` : beforeunload·pagehide 이벤트)이나 권한을 끈 기록이 있으면 '장애 아님', 나머지는 모두 '장애'(원인 모름 포함, 지원자에게 유리하게).
- 실시간 연결(WebSocket)이 "정상 종료"를 뜻하는 번호(1000·1001)로 닫혀도, 창 닫기 기록이 함께 있을 때만 지원자가 끈 것으로 본다.
- 다시 접속하면 장치 점검 API를 다시 통과해야 하고, 끊긴 질문을 처음부터 새 답변 시간으로 다시 낸다. 이어가기 기한은 공고 마감 시각까지(PRD INT-10). 자리를 이미 돌려준 뒤라도 N을 다시 확인하지 않고 들어온다(5-3).

장애 유형별 감지 기준

PRD 종류	알아채는 신호	판단
인터넷 끊김	heartbeat 3번 연속(15초) 안 옴, WebSocket이 비정상으로 닫힘(종료 번호가 1000·1001이 아니거나, 창 닫기 기록 없이 1000·1001), 브라우저 `offline` 이벤트, 지원자 IP가 바뀌어 다시 연결됨	장애
브라우저·기기가 갑자기 꺼짐	직전 30초 안에 `unload` 기록 없이 연결이 끊김(브라우저 멈춤, 재시작, 배터리 방전, 강제 업데이트)	장애
장치 오류	마이크·카메라 스트림이 끝남(`MediaStreamTrack` 의 `ended` 이벤트 — 장치 빠짐·드라이버 오류), 마이크 입력이 10초 넘게 0. 지원자가 권한을 끈 기록(`permission`)이 함께 있으면 제외	장애
업로드 실패	답변 음성·코드 올리기를 `upload_retry_limit` (3번)만큼 다시 해도 실패	장애
서버·AI 오류	API 5xx 오류, STT·TTS 서버가 답이 없음, 음성 글자 바꾸기를 `stt_retry_limit` (2번) 다시 해도 실패. 질문 만들기 실패는 7-4 ⑤로 처리하고 장애가 아니다	장애
원인 모름	위 신호로 알 수 없음. 창 닫기 기록이 서버에 못 온 경우(sendBeacon 전송 실패 등)도 여기에 들어간다	장애(지원자에게 유리하게)
스스로 종료	탭 닫기·새로고침·뒤로 가기·주소 이동 때 `beforeunload` · `pagehide` 이벤트를 `navigator.sendBeacon` (창이 닫혀도 보내지는 전송 방법)으로 서버에 남긴 뒤 연결이 끊김	장애 아님
권한 끄기	브라우저 권한 상태 변경 이벤트로 마이크·카메라 권한이 '거부'로 바뀜	장애 아님

7-3. 화면 이탈 감지

- 화면: 탭이 숨겨짐(`visibilitychange` hidden), 창이 포커스를 잃고(`blur`) `blur_threshold_sec` (2초) 넘게 안 돌아옴, 전체 화면이 꺼짐(`fullscreenchange`), 다른 주소로 가려 함 — 이것들을 서버에 보낸다. 서버가 횟수를 세고, 면접과 코딩테스트를 합쳐 센다.
- 장애로 판단된 구간 안에서 생긴 이탈 신호는 세지 않는다. 2초가 안 되는 포커스 잃음은 화면에서 버리고 보내지 않는다.
- **1번째 이탈 처리 (CHT-02)**: 서버는 이탈 신호를 받은 순간 그 질문의 남은 답변 시간을 저장하고 시간을 멈춘다. 화면을 벗어나 있는 동안과 경고 창이 떠 있는 동안에는 시간이 흐르지 않는다. 지원자가 경고 창에서 확인을 누르면 전체 화면으로 돌아가 **같은 질문을 저장한 남은 시간부터 이어서** 답한다(답변 시간을 새로 주지 않음). 이탈 전까지 받은 답변 음성은 지우지 않고, 이어서 받는 음성을 뒤에 붙여 하나의 답변으로 글자로 바꾸고 채점한다(7-5). 코딩테스트도 같은 방식으로 남은 시간 그대로다.

- 2번째 이탈은 바로 '응시 취소'로 끝내고 자리를 돌려준다(CHT-03).
- 눈동자·얼굴 분석은 만들지 않는다(CHT-06).

7-4. 질문 생성 절차

②~⑤는 worker가 LangGraph 그래프 하나로 돌린다. 질문 자리마다 아래 순서로 지나간다.

1. **고르기**: 질문 DB에서 후보를 고른다(②).
2. **만들기**: 질문 DB에서 비슷한 질문을 찾아 예시로 넣고(RAG), LangChain으로 LLM을 불러 질문을 만들거나 다듬는다(③).
3. **확인하기**: 범위를 확인하고, 형식·내용이 맞는지 코드가 검사한다(④).
4. **갈림길**: 통과하면 저장한다. 실패하고 다시 만들기 횟수가 남았으면 2번으로 돌아간다. 횟수를 다 썼거나 60초 제한에 걸리면 대신 쓸 질문으로 바꾼다(⑤).

단계	처리
① 질문 DB 불러오기	IT직무·인성 엑셀을 질문 DB에 넣고, BGE-M3로 질문마다 숫자 목록(임베딩)을 만들어 둔다. 이 숫자로 뜻이 비슷한 질문을 찾는다. **불러오기 규칙**: IT직무_면접질문.xlsx는 시트 32개(기업_ 시트 18개 6,577행, 생성_ 시트 13개 137행, 빈 예시 시트 1개는 뺌). 직무 질문은 머신러닝·딥러닝·컴퓨터 비전·데이터 사이언스·파이썬 관련 시트만 쓰고, 나머지 시트는 준비 중 직무용으로 넣어만 둔다(status='preparing'). 인성_면접질문.xlsx는 448개 질문·21개 종류이며, 지원동기·지원 이유·조직 문화 적합성 종류는 '회사 관련', 나머지는 '인성'으로 나눈다. '다른 중복 시트' 값으로 duplicate_group(겹치는 질문 묶음)을 채우고, '코딩문제여부'가 '예'인 질문과 질문 글이 빈 행은 말로 하는 면접에서 뺀다. 엑셀에 없는 answer_seconds(90초)와 eval_items(PRD 5-7의 종류별 평가 항목)는 불러올 때 붙인다. 불러오기는 `manage.py load_questions` 명령으로 하며, 같은 파일을 다시 넣어도 (시트, 행 번호)로 구분해 겹치지 않는다
② 질문 고르기	종류별 조건(PRD 5-5 표)으로 후보를 뽑고, **그 공고에서 덜 쓰인 질문**(공고별 use_count가 작은 순)부터 고른다. 쓰인 횟수가 같으면 지원 번호를 기준으로 한 무작위 순서로 정해, 같은 지원 건은 다시 돌려도 같은 결과가 나오게 한다. 한 면접 안에서 같은 duplicate_group이거나 숫자 목록으로 잰 비슷한 정도(코사인 유사도, 1에 가까울수록 비슷함)가 0.9 이상인 질문은 하나만 쓴다. use_count는 질문 사본이 확정될 때 1 올린다(응시하지 않아 지워지는 구성분은 되돌림)
③ 만들기·다듬기	이력서 기반 2개는 면접 중 LLM이 이력서 사본을 보고 만든다. 회사 관련 2개는 공고 본문에 맞게 말만 바꾼다. 후보가 모자라면 같은 종류 질문을 새로 만든다(QST-02). 새로 만드는 질문은 아래 순서로 예시를 찾아 넣는다(RAG). 1. **찾을 글 만들기**: 이력서 기반은 이력서 사본 요약, 새로 만드는 질문은 공고 본문과 질문 종류를 BGE-M3로 숫자 목록(임베딩)으로 바꾼다. 2. **비슷한 질문 찾기**: pgvector로 질문 DB에서 같은 종류이고 뜻이 가장 가까운 질문을 `rag_example_count`(기본 3)개 꺼낸다. 이 면접에 이미 고른 질문은 뺀다. 3. **지시문에 넣기**: 꺼낸 질문을 "이런 수준과 말투로 쓰되 그대로 베끼지 말 것"이라는 지시와 함께 예시로 넣는다. 4. **베낌 확인**: 만든 질문이 예시 중 하나와 비슷한 정도(코사인 유사도) 0.9 이상이면 베낀 것으로 보고 실패로 센다(⑤).
④ 범위 확인	LLM이 {purpose 평가 목적, task_count 요구하는 일의 수, fits_time 시간에 맞는지, revised_text 줄인 질문} 형식의 JSON을 돌려준다. 요구하는 일의 수가 답변 시간 기준(90초면 2개 이하)을 넘으면 줄인 질문으로 바꾼다(QST-04)
⑤ 실패 처리	JSON 형식 오류, 확인 실패(이력서에 없는 내용 등), 1번 시도에 8초 넘김(`question_attempt_timeout_sec`)은 실패로 보고, 같은 입력으로 최대 `question_regen_limit`(기본 3)번 다시 만든다. 횟수는 질문 자리마다 따로 센다. 지원자 1명의 질문 전체를 만드는 데 `question_compose_budget_sec`(기본 60초) 제한을 둔다. 제한에 걸리면 진행 중인 시도를 끊고, 남은 자리는 LLM을 부르지 않고 바로 다른 질문으로 채운다. 3번 다 실패하면 ②의 후보 중 확인된 DB 질문으로, 그것도 없으면 종류별 예비 질문(source='reserve', 관리자가 미리 확인)으로 바꾼다. 이력서 기반 자리는 맞는 DB 종류가 없어서 바로 예비 질문의 '경험·프로젝트' 종류에서 고른다. 다시 만든 횟수·바꿨는지·이유를 기록하고, 장애로 보지 않는다
⑥ 미리 준비(만드는 시점)	이력서가 확정되면 worker의 'compose' 작업 줄(순서가 뒤로 밀리는 줄)에 질문 만들기 일을 넣는다. 진행 중인 면접 요청이 먼저 처리되도록 llama.cpp의 동시 요청 자리를 면접용과 질문 만들기용으로 나눈다. 만들기용 자리 수는 설정값 `compose_slots`(처음 값 1)이고, 13-2에서 그래픽카드 여유를 보고 정한다(6-2 계산에 함께 들어감). ②~⑤가 끝나면 10개를 interview_questions에 저장하고 질문 음성(TTS)도 미리 만든다. 시작 버튼을 눌렀는데 아직 다 못 만들었으면

	'질문 준비 중'으로 기다렸다 시작한다. 이 기다림에도 `question_compose_budget_sec`(60초)를 상한으로 건다: 시작 버튼을 누른 뒤 60초가 지나면 진행 중인 시도를 끊고, 아직 못 만든 자리는 바로 ⑤처럼 질문 DB 질문, 그다음 예비 질문으로 채워 시작한다. 이력서 확정 뒤 다른 사람 차례를 기다리는 시간은 지원자가 겪지 않으므로 따로 세지 않는다. 응시 횟수는 첫 질문이 나올 때 쓰므로, 질문만 만들고 응시하지 않으면 횟수를 쓰지 않는다. 쓰지 않은 질문 구성은 응시 데이터 보관 규칙대로 지운다

7-5. 답변 처리 및 꼬리질문

면접 진행 그래프 (LangGraph)

api가 지원 1건마다 그래프 하나를 돌린다. 그래프의 이름표(thread_id)는 지원 번호다.

1. **질문 내기**: 다음 질문(미리 만든 기본 질문 또는 꼬리질문)을 WebSocket으로 보낸다.
2. **답변 기다리기**: 그래프를 멈춰 두고(interrupt), 답변 음성이 다 올라오면 그 자리에서 다시 움직인다.
3. **글자로 바꾸기**: STT로 답변을 글자로 바꾼다.
4. **꼬리질문 판단**: 남은 시간이 있으면 LangChain으로 LLM을 불러 판단 재료를 받는다.
5. **갈림길**: 코드가 정한다. 꼬리질문이면 1번으로, 기본 질문이 남았으면 다음 질문으로 1번, 10개를 다 했으면 코딩테스트로, 마감·응시 취소면 끝으로 간다.

그래프가 단계를 지날 때마다 진행 상태(지금 질문 번호, 꼬리질문 수, 남은 답변 시간)를 PostgreSQL에 저장한다(LangGraph 체크포인트). 장애로 끊겼다 다시 접속하면 같은 지원 번호로 저장된 상태를 불러와 멈춘 단계부터 이어 간다(7-2). 응시 상태(7-1)는 지금처럼 applications 테이블이 기준이고, 체크포인트는 그래프를 이어 가는 데만 쓴다.

- 답변 음성은 WebRTC로 실시간으로 받고, 답변 중에는 임시 자막을 보여 준다(5-2). 답변이 끝나면 faster-whisper(한국어, 소리 없는 부분 거르기)로 전체 음성을 최종 글자로 바꾼다. 실패하면 2번 다시 하고, 그래도 안 되면 장애로 처리한다.
- 내용 없는 답 판단: 바꾼 글자가 비었거나 '모르겠습니다' 같은 말이면 꼬리질문 없이 다음 질문으로 간다(INT-07).
- **시간 나눠 쓰기**: 답변이 끝나고 다음 질문이 뜨기까지의 목표 시간은 설정값 `next_question_target_sec`이고, 처음 값은 13-2에서 운영 서버로 재서 정한다. 그 안에서 ① 음성을 글자로 바꾸기(STT) ② 꼬리질문 판단 ③ 꼬리질문 음성 만들기(TTS)를 차례로 한다. 기본 질문 음성은 미리 만들어 두었으므로, 꼬리질문이 없으면 ②가 끝나는 대로 다음 질문을 보낸다.
- 꼬리질문 판단에는 고정된 제한 시간을 따로 두지 않고, 답변마다 **남은 시간** = 목표 시간 − ①에 실제로 걸린 시간 − 꼬리질문 음성에 걸리는 시간(최근 평균)을 계산해 그만큼만 준다. 서버가 바쁠 때는 판단 시간이 저절로 줄고 한가할 때는 늘어난다. 남은 시간이 `followup_min_sec`보다 짧으면 판단하지 않고 꼬리질문 없이 넘어간다.
- 꼬리질문 판단은 면접 중 LLM에 남은 시간을 제한으로 걸고 불러 {need 필요한지, reason_code 이유, question 꼬리질문} JSON을 받는다. 시간을 넘기거나 형식이 틀리면 꼬리질문 없이 넘어간다. 질문당 1개, 전체 4개 제한은 서버가 센다.
- 꼬리질문이 나오면 CPU의 TTS로 바로 음성을 만든다. 목표 시간 안에 음성이 안 만들어지면 그 꼬리질문은 음성 없이 자막만으로 낸다(question 메시지의 음성 주소 null, INT-01).
- 1번째 화면 이탈로 끊겼던 답변(7-3)은 이탈 전과 뒤에 받은 음성을 이어 붙여 한 번에 최종 글자로 바꾼다.

7-6. 이력서 내용 추출 및 OCR

단계	처리
① 올린 파일 확인	PDF·JPG·PNG만 받는다. 사용자에게 알려 주는 용량 제한은 없다. 확장자와 파일 앞부분의 실제 형식 표시를 함께 확인한다(이름만 바꾼 파일 거르기). 서버를 지키기 위한 요청 크기 한도는 설정값 `upload_max_mb`(api가 직접 확인, 사용자에게는 안 보임) 하나만 둔다. 올리자마자 예상 처리 시간 = 글자 있는 쪽 수 × 0.5초 + 스캔 쪽 수 × 쪽당 OCR 시간(13-2에서 재서 정함, 처음 값 8초) + LLM 정리 시간(`resume_llm_sec` , 13-2에서 재서 정하고, 그 뒤로는 최근 처리 시간 평균으로 계속 맞춤)을 계산해 화면에 보여 준다. 원본은 MinIO에 암호화해 저장
② 글자부터 꺼내기	PDF는 pdfplumber로 쪽마다 글자를 꺼낸다. 한 쪽에서 쓸 만한 글자가 50자 미만이면 그 쪽은 스캔한 쪽으로 보고 ③으로 보낸다

③ OCR	스캔 쪽과 사진 파일을 pypdfium2/Pillow로 300dpi 이미지로 만들고, 기울어진 것을 바로잡은 뒤 PaddleOCR(한국어, CPU)로 글자와 줄마다 "얼마나 확실한지"(신뢰도)를 얻는다. worker의 'ocr' 작업 줄에서 뒤에서 처리한다. 걸린 시간이 예상 시간의 2배(최소 60초)를 넘거나, 읽은 글자가 없거나, 평균 신뢰도가 0.5 미만이면 실패로 보고 "불러오기 실패 — 직접 입력으로 진행할까요?"라는 답을 돌려준다
④ 항목별 정리	뽑은 글자를 채점용 LLM에 넣어 학력·경력·프로젝트·기술·자격증 JSON으로 정리한다. 채점용 LLM은 미리 채점(8-2 ①)과 같이 쓰므로 작업 순서를 정한다: 지원자가 화면 앞에서 기다리는 이력서 정리를 먼저 하고, 미리 채점은 그다음에 한다(Celery 작업 우선순위). 미리 채점은 결과 10분 목표까지 여유가 있다. OCR 신뢰도가 0.80 미만인 줄에서 나온 칸은 ocr_low_conf_fields에 넣어 화면에서 강조한다
⑤ 확인·수정	지원자가 결과를 확인·수정하고 확정하면 resume_snapshot(이력서 사본)으로 저장한다(APP-05). OCR 실패나 글자가 없으면 직접 입력 화면으로 보낸다
⑥ 보관·삭제	OCR 중간에 만든 이미지는 처리 직후 지운다. 뽑은 글자와 원본은 면접 데이터로 보고 공개일 + 7일에 지운다(PRD 10-2)

OCR은 면접 시작 전에 끝나므로 면접 중 그래픽카드 메모리에 영향을 주지 않는다. OCR 모델 파일도 미리 받아 우리 저장소에 둔다(밖으로 나가는 데이터 0건).

8. 채점 및 1차 합격 결정

8-1. 채점 흐름

① 질문 번호마다 미리 채점
채점 LLM · 항목별 0~5점 + 근거 문장
→
② 근거·형식 확인
근거가 틀리면 1번 다시 받기 → 그래도 틀리면 1점
→
③ 점수 계산
항목 점수 · AI 면접 점수 · 총점
→
④ AI 합격 여부 결정
결정 + 이유 + 근거 문장
→
⑤ 결과 검증
형식 · 값 범위 · 근거
→
⑥ 확정 · 리포트
고칠 수 없음

8-2. 단계별 구현 방식

단계	만드는 방법
① 미리 채점	**그 질문 번호가 끝났을 때** worker가 채점 LLM을 부른다. 꼬리질문이 없다고 정해지면 기본 질문 답변이 끝난 바로 그때, 꼬리질문이 있으면 꼬리질문 답변까지 끝난 뒤 둘을 합쳐 채점한다(아직 나오지 않은 꼬리질문 답변은 합칠 수 없으므로). 넣는 것: 질문 사본, 답변 글자(꼬리질문 답변 합침), 해당 평가 항목 설명과 채점 기준표, 이력서 사본 요약. 이름·연락처·목소리 특징은 넣지 않는다. 받는 형식: `{items:[{item 항목, score 점수, quote 근거 문장}]}`. llama.cpp의 형식 강제 기능(grammar)으로 이 JSON 형식이 아니면 아예 나오지 않게 한다
② 확인	점수가 0~5 정수인지, 근거 문장이 답변 원문 안에 정말 있는지(띄어쓰기 차이는 무시하고) 확인한다. 틀리면 1번 다시 받고 (`quote_regen_limit`), 그래도 틀리거나 답은 했는데 근거가 없으면 1점, 답을 안 했으면 0점
③ 계산	항목 점수 = 그 항목 질문 점수들의 평균 ÷ 5 × 100, AI 면접 점수 = (항목 점수 × 항목 비율)의 합, 총점 = 반영 비율로 합침(코딩테스트 안 쓰면 AI 면접 점수). Decimal로 계산하고 소수점 둘째 자리까지
④ AI 결정	채점 LLM에 항목 점수·총점·합격 기준·항목별 근거 문장을 주고 `{decision:"pass"|"fail", reason 이유, quotes[] 근거 문장들}` 를 받는다. 지시문에 "총점·합격 기준·항목별 근거를 보고 정하고, 이유는 근거 문장으로 설명하라"고 적는다
⑤ 결과 검증	(a) 형식이 맞는지 (b) 값이 정해진 범위인지(decision은 pass/fail 중 하나, 이유가 비어 있지 않음) (c) 근거 문장이 원문에 있는지. 합격 여부를 서버가 "총점 ≥ 합격 기준" 같은 별도 규칙으로 다시 판정하지 않는다. 하나라도 틀리면 1번 다시 정

	하게 하고(`rejudge_limit`), 그래도 틀리면 analysis_status='needs_review'(확인 필요)로 바꿔 관리자에게 알리고 다시 돌리게 한다(EVL-08). 확인을 통과한 결정만 확정한다
⑥ 확정	results에 결정·이유·점수 구성·채점 기준 버전·모델·지시문·채점 기준표 버전을 저장하고, 관리자용 리포트와 지원자용 결과(public_view: 합격 여부·강점·개선점·상태만)를 따로 만든다

8-3. 실패·지연 처리

- 뒤에서 도는 작업(Celery)마다 중복 방지 표시(멱등 키: 지원 번호 + 단계)를 붙여, 같은 작업이 두 번 돌아도 결과가 한 번만 생기게 한다. 실패하면 1·2·4분 간격으로 최대 3번 다시 하고, 30분이 넘거나 3번 다 실패하면 'needs_review'(확인 필요)로 바꾼다.
- 5분마다 "응시는 끝났는데 채점 작업이 없거나 멈춘" 건을 찾아 다시 넣는다(AC-17).
- 부정행위로 취소된 건은 채점하지 않고 decision='fail', reason='응시 취소'로 확정한다.

8-4. 하이라이트 구간 선정

- 후보: 확정된 항목 점수 중 4~5점(잘함)과 1~2점(깎임). 질문마다 가장 높은 점수 1개와 가장 낮은 점수 1개를 고르고, 응시 1건에 잘함·깎임 각각 최대 3개를 점수 차가 큰 순서로 남긴다. 0점(답 없음)은 들을 구간이 없으므로 뺀다.
- 구간 위치: 근거 문장을 STT가 준 단어별 시각(faster-whisper `word_timestamps=True`)과 맞춰 시작·끝 시각을 구하고, 앞뒤로 1초씩 늘린다. 근거 확인(8-2 ②)을 통과한 문장만 쓰므로 위치를 못 찾는 일은 없다. 근거 없음으로 1점이 된 항목은 그 답변 전체를 구간으로 한다.
- 저장: highlights 테이블(4-1). 음성 원본과 같이 공개일 + 7일에 지운다.
- 관리자 화면은 답변 음성 시간 막대 위에 구간을 표시하고, 누르면 그 시각부터 재생한다(MinIO가 만들어 주는 5분짜리 임시 주소로 재생).

9. 코딩테스트 실행 환경

항목	값
격리	실행할 때마다 새로 만들고 버리는 상자(python:3.11-slim), 인터넷 없음(`--network none`), 시스템 파일은 읽기만 가능, 관리자 권한 없는 사용자로 실행, 프로세스 64개 제한(`--pids-limit 64`)
제한	문제당 실행 10초(실제 시간), 메모리 256MB, CPU 1개
채점	공개 + 숨겨진 테스트 통과 비율 × 100, 2문제 같은 배점. 숨겨진 테스트와 정답은 실행 상자 안에만 넣고, API 응답이나 AI 입력에는 넣지 않는다
편집기	Monaco, 30초마다 자동 저장, 밖에서 복사한 내용 붙여넣기 막기(붙여넣기 이벤트 때 편집기 안에서 복사한 내용과 같은지 비교), 시도 횟수 기록
시간	2문제 합쳐 60분, 시간은 서버 기준으로 잰다. 장애로 끊긴 시간은 빼지 않는다. 끝나면 마지막 저장 코드를 자동 제출

10. 결과 공개·알림·자동 작업

작업	언제	내용
공통	—	모든 정해진 시각 작업과 날짜 계산은 한국 시간(`Asia/Seoul`) 기준. 지원 건마다 applications.release_at(공개 시각)과 purge_at(삭제 예정 = 공개 시각 + 7일)을 저장해, 한꺼번에

		공개하는 건과 늦게 공개하는 건을 같은 방식으로 처리한다. 녹화 삭제 시각은 응시 끝 + 3일로 따로 계산한다
결과 한꺼번에 공개	매일 10:00	release_at이 지난 확정 결과를 '공개'로 바꾸고 사이트 알림 + 이메일. 결과가 늦게 나온 건은 나온 때 release_at을 다음 날 10:00으로 정해 따로 공개
보관 기간 지난 데이터 삭제	매일 03:00	녹화: 응시 끝 + 3일, 면접 데이터: purge_at(공개 + 7일)이 지난 건. 원본과 거기서 만든 데이터(LangGraph 체크포인트 포함)를 지우고 deletion_logs에 기록
빠진 작업 찾기	5분마다	멈추거나 빠진 채점 작업을 다시 넣기. 같은 주기에 동시 면접 수(Redis 숫자)를 DB의 holds_seat 표시 수로 다시 맞추고(5-3), 끊긴 지 10분이 지난 '장애 중단' 면접의 자리를 돌려준다
공고 마감 처리	마감 시각(close_at) 정각	게시할 때 마감 시각에 맞춰 예약 작업(Celery eta)을 걸어 두고, 5분마다 점검해서 빠진 것을 보완한다. 마감되면: 공고 상태 '마감', 준비 중 응시 → 'not_taken_closed'(미응시 마감), 진행 중·장애 중단 면접 → WebSocket으로 `terminate{reason:'closed'}` 를 보내 끝내고 'ended_closed'(응시 종료 마감)로 바꾼 뒤 채점 작업 넣기. 마감 뒤 시작·이어가기 API는 409와 안내 코드. 동시 면접 수에서 자리 돌려주기
알림·안내	매시	마감 3일·1일 전 아직 응시 안 끝낸 지원자, 마감 1일 전 관리자, 결과 보기 종료 2일 전 지원자(건별 purge_at 기준). 관리자에게 보내는 '삭제 2일 전' 알림은 건마다 날짜가 다르므로 **날짜별로 묶어** 보낸다: 매일 09:00에 삭제 예정이 2일 뒤인 지원 건을 공고별로 모아 한 통으로 알린다
메일 다시 보내기	실패했을 때	Celery 작업으로 최대 3번 다시 보낸다(60초·300초·1800초 뒤). 메일 서버가 "잠시 뒤 다시"(4xx)라고 하거나 연결이 안 될 때만 다시 보내고, "주소 없음" 같은 영구 오류(5xx)는 바로 실패 처리한다. 끝내 실패하면 notifications.status='failed'와 이유를 기록하고, 관리자 대시보드에 실패 건수를 보여 준다. 사이트 알림은 메일과 상관없이 항상 만든다
이메일 인증	요청할 때	나중에 붙이는 기능. 만들 때는 24시간짜리 인증 링크 메일을 보내고(다시 보내기는 1분에 1번), 인증 시각을 email_verified_at에 기록하고, 설정 `email_verification_required` 를 true로 바꿔 지원 API에서 확인한다. 만들기 전에는 false. 추천안 켤 때 기존 가입자: 이미 진행 중인 지원은 확인하지 않고, **새 지원 만들기(POST /applications)**에서만 인증 여부를 본다. 미인증이면 403과 인증 메일 다시 보내기 안내

11. 보안·개인정보

분야	어떻게 만드나
주고받는 데이터	브라우저와 서버가 같은 PC 안(127.0.0.1)에서만 주고받으므로 데이터가 네트워크 밖으로 나가지 않는다. 그래서 HTTPS 암호화는 쓰지 않는다. 서버끼리 통신은 내부망 안에서만 한다. 다른 PC에서 접속하게 바꾸면 그때 HTTPS(TLS 1.2 이상)를 넣는다
로그인 후 돌아갈 주소	ACC-04에서 로그인 뒤 돌아갈 주소(`next`)는 '/'로 시작하는 우리 사이트 안 주소만 받는다. '//'나 '/\'로 시작하거나 http·다른 도메인이 들어 있으면 안 받는다. 허용하는 주소 앞부분은 `/postings/` , `/applications/` , `/me` . 안 맞으면 `/me` 로 보낸다(로그인 화면을 이용해 가짜 사이트로 보내는 공격 막기)
저장한 데이터	MinIO 파일은 저장할 때 암호화(AES-256), DB의 민감한 칸은 서버 코드에서 AES-256-GCM으로 암호화한다. 암호화 키는 환경 변수가 아닌 별도 키 파일에 두고, 서버 관리자만 읽을 수 있게 권한을 600으로 둔다

로그인	비밀번호 규칙(PRD 5-1): 8~64자, 영문·숫자·특수문자 각 1자 이상, 이메일 아이디 포함 금지, 같은 문자 4번 이상 연속 금지. 가입·변경·재설정 API에서 서버가 확인한다. 비밀번호는 Argon2id로 바꿔 저장하고, 로그인 상태는 쿠키로 유지한다(HttpOnly: 화면 스크립트가 못 읽음, Secure: HTTPS로만 — 이 PC 안에서 http로 실행하는 지금은 설정값 `cookie_secure`로 끔, SameSite=Lax: 다른 사이트에서 함부로 못 씀). 마지막 요청 후 2시간 지나면 만료(면접 중에는 계속 늘림), 5번 틀리면 10분 잠금, 재설정 링크 30분
권한	역할과 데이터 주인을 확인한다. 관리자 API(`/admin/*`)에는 관리자 확인(`require_admin`)을 붙이고, 지원자 API에는 주인 확인(`require_owner`)을 붙여 모든 조회에 "applicant_id = 지금 로그인한 사람" 조건을 넣는다(이력서·답변·결과·코드 제출·녹화 주소 포함). 다른 지원자의 데이터 번호로 요청하면, 그런 데이터가 있는지조차 알 수 없게 404를 준다. 관리자 API에서 결과·결정은 읽기만 된다(메모·설정·다시 돌리기만 예외). 회사별 확인은 없다(1-3)
로그·배포	모델·프로그램 이미지는 우리 저장소에서 배포하고, 이름표(태그)와 고유값(해시)으로 버전을 고정한다. 감사 로그 대상: 결과 보기, 녹화 다시 보기, 메모, 동의, 기준 변경, 설정 변경, 삭제, 장애·이탈 판단. 프로그램 로그에서는 답변 원문과 개인정보를 가린다
밖으로 나가는 것 막기	AI·데이터 상자는 밖으로 나가는 길이 없는 내부망에만 연결해서, 면접 데이터가 밖으로 나갈 수 없는 구조로 만든다(AC-19). 모델 파일은 미리 받아 우리 저장소에서 연결한다. 밖으로 나갈 수 있는 상자는 api(Open API 공고 불러오기)와 mailer(팀 Gmail SMTP, smtp.gmail.com 587번 포트)뿐이고, mailer는 그 메일 서버 말고는 나갈 수 없게 막는다. 메일에는 알림 문구와 링크만 넣고 답변·점수·이력서 내용은 넣지 않는다. Gmail 앱 비밀번호는 암호화 키처럼 서버 설정 파일에만 두고 저장소에 올리지 않는다
공정성	AI에 넣을 채점 자료를 만드는 함수에서 개인 정보 칸을 빼는지 확인하는 테스트를 둔다. 이름·성별만 바꾼 같은 답변으로 점수 차가 ±0.5점 안인지 출시 전에 시험한다

12. 성능 목표 및 측정

무엇	목표	어떻게 재나
답변 끝 → 다음 질문 표시	100번 중 95번은 `next_question_target_sec` 안 (13-2에서 재서 정함)	서버가 '답변 완료'를 받은 시각과 다음 질문을 보낸 시각의 차이
꼬리질문 판단	목표 시간에서 STT·꼬리질문 음성 시간을 뺀 남은 시간 안, 넘으면 생략(7-5)	답변마다 계산한 LLM 호출 시간 제한
일반 API	100번 중 95번은 `api_p95_target_sec` 안 (13-2에서 재서 정함)	모든 요청의 응답 시간 기록
결과 나오는 시간	응시 끝 → 결과 확정, 10건 중 9건은 10분 안	completed_at(응시 끝)과 decided_at(결정) 차이
음성 → 글자 정확도	틀린 단어 비율 10% 미만	한국어 면접 답변 30개를 사람이 받아 적은 것과 비교
그래픽카드 메모리	동시 면접 N개를 진행할 때 5.4GB 이하, 일반 메모리 빌려 쓰기 0	NVML(그래픽카드 상태 읽기)로 기록
AI 점수 일치도	정답 세트로 비교해, 두 채점이 얼마나 같은지 재는 지표(가중 카파)가 0.6 이상, ±1점 이내 80% 이상	종류별 30개 답변을 전문가 2명이 채점한 것과 비교

13. 테스트 계획

13-1. 완성 기준별 테스트 방법

AC	테스트 방법
AC-01~05	API 테스트: 로그인 안 하고 부르면 401, 지원자 A로 로그인해 지원자 B의 이력서·답변·결과·코드 제출 번호를 부르면 404, 지원자로 /admin을 부르면 403, 가입 칸 확인, 준비 중 직무 선택 거절, 두 번째 응시 거절, 준비가 안 끝났으면 시작 거절
AC-06~08	질문 만들기 테스트(10개 구성·꼬리질문 최대 개수), QST-04 세 가지 경우, 화면에 중단 버튼이 없는지(실제 브라우저로 처음부터 끝까지 눌러 보는 E2E 테스트)
AC-09~11	브라우저 자동 조작 도구(Playwright)로 E2E 테스트: 인터넷 끊기 → 15초 안에 장애 중단 → 다시 접속해 이어가기를 여러 번, 끊긴 채 10분을 넘긴 뒤(시각 넘기기) 자리가 돌아왔는지와 그 뒤 다시 접속해도 들어와지는지, 탭 닫기 → 15초 시점에 바로 응시 종료·자리 돌려줌, 탭 바꾸기 1번 → 돌아오면 같은 질문을 멈춘 시점의 남은 시간부터 이어 가고 앞뒤 답변이 합쳐져 저장되는지, 탭 바꾸기 2번 → 응시 취소
AC-12~17	채점 처리 테스트: 근거 틀림·근거 없음 처리, 손 계산과 비교, 형식·근거가 틀린 AI 결정을 일부러 넣어 다시 결정·확인 필요로 가는지 확인, 결정 수정 API가 없는지, 기준 잠금, 공개 전에 안 보이는지, 빠진 작업 찾기
AC-18~19	시각을 바꿔(freezegun) 삭제 작업 확인, 내부망에서 밖으로 접속이 안 되는지 확인
AC-20 (QST-05)	가짜 AI 응답(mock)으로 3번 연속 실패를 넣어 질문 DB의 질문으로 바뀌는지, 질문 DB 후보를 없애고 예비 질문으로 바뀌는지, 기록 확인
AC-21 (INT-08)	N=1로 두고 두 번째 시작이 429·횟수 안 씀인지, 설정을 바꾸자마자 적용되는지 확인. Redis 숫자를 일부러 틀리게 바꿔 두고 다시 맞추기 작업이 DB의 holds_seat 수로 고치는지 확인
AC-22 (APP-04)	스캔 PDF·사진 이력서 10종(기울어짐·저해상도 포함)으로 뽑은 결과와 신뢰도 표시 확인, 처리 중 밖으로 나가는 요청 0건. 시간 초과·저신뢰 샘플로 '불러오기 실패 → 직접 입력' 흐름 확인
AC-23 (INT-10)	시각을 마감 뒤로 넘겨(freezegun) 준비 중·면접 진행 중·코딩테스트 진행 중·장애 중단 각 1건을 확인: 시작 전 건은 not_taken_closed가 되고 시작 API 409·버튼 막힘 문구, 진행 중 건은 terminate{reason:'closed'} 뒤 ended_closed·채점 작업 등록·남은 질문 0점, 장애 중단 건은 이어가기 409, 동시 면접 수 자리 돌려주기

13-2. 운영 서버 측정 계획

1. 모델을 하나씩 올려 메모리를 잰다: Qwen3-4B Q4_K_M(그래픽카드), Whisper Turbo int8_float16(그래픽카드), Qwen3-8B Q4_K_M(CPU).
2. 질문 만들기 자리(`compose_slots`)가 돌고 있는 상태에서, 가짜 지원자 스크립트로 동시 면접을 1, 2, 3, 4…로 늘려 가며 '다음 질문' 시간(100번 중 95번 기준), STT 시간, 꼬리질문 판단 시간, 꼬리질문 음성(TTS) 시간, 그래픽카드 메모리를 기록한다. 지원자가 기다림을 느끼지 않을 만한 다음 질문 목표 시간(`next_question_target_sec` , 처음 잴 때 기준 5초)과 꼬리질문을 판단할 최소 시간(`followup_min_sec`)을 정하고, 그 목표와 5.4GB를 지키는 가장 큰 값을 N의 처음 값으로 정한다. 일반 API 응답 시간도 같이 재서 `api_p95_target_sec` (처음 잴 때 기준 0.3초)를 정한다.
3. 동시 면접 수를 늘려 가며 실시간 자막이 늦지 않는 `caption_interval_sec` 를 찾는다. 자막 때문에 '다음 질문' 시간이 목표를 넘으면 간격을 늘린다.
4. 이력서 샘플로 OCR 쪽당 시간과 LLM 정리 시간을 재서 `ocr_sec_per_page` 와 `resume_llm_sec` 의 처음 값으로 둔다.
5. 응시 1건(질문 10개 + 꼬리질문 4개)의 채점·결정을 8B CPU로 돌려 시간을 잰다. 10분을 넘으면 6-4의 방법 B나 C로 바꾼다.
6. 결과를 이 문서 6장과 14장 설정값에 반영하고, 바뀐 내용은 TRD_변경이력에 남긴다.

14. 설정값 목록

1. **한 곳에서 관리**: 재시도 횟수·시간 제한·기준값·주기는 이 표에만 두고, 본문은 이 표를 가리킨다. 값은 `app_settings`(서버 재시작 없이 바꿈)나 환경 설정 파일에 둔다.
2. **운영 서버 열**: 기준값이다. 최종 테스트와 완성 기준 확인은 이 값으로 한다(PRD 10-1).
3. **개발 환경 열**: 팀원 각자 컴퓨터(작은 그래픽카드나 CPU만 있음)에서 기능만 확인할 때 쓰는 느슨한 값이다.
4. **"13-2로 정함"**: 서버의 실제 빠르기에 달린 값이다. 13-2에서 운영 서버로 재서 넣는다. 괄호 안 숫자는 처음 잴 때 기준으로 삼는 값이다.

키	운영 서버	개발 환경	뜻	본문
면접 진행				
`max_concurrent_interviews`	2 (임시, 13-2로 확정)	1	동시 면접 상한 N	5-3
`disconnect_seat_hold_min`	10분	같음	장애로 끊긴 면접의 자리를 맡아 두는 시간. 지나면 자리만 돌려줌(이어가기 기한은 마감 시각)	5-3, 7-2
`seat_reconcile_interval_min`	5분	같음	동시 면접 수를 DB의 holds_seat 수로 다시 맞추는 주기	5-3, 10장
`admission_guard_vram_pct` · 점검 주기	92% · 30초	끔	새 입장을 잠시 막는 조건(메모리 쪽). 시간 쪽 기준은 `next_question_target_sec`를 같이 씀	5-3
`heartbeat_interval_sec` · `heartbeat_miss_limit`	5초 · 3번(15초)	같음	연결 끊김 알아채기(PRD AC-09의 15초)	5-2, 7-2
`unload_lookback_sec`	30초	같음	끊기기 직전 '창 닫기' 기록을 찾아보는 시간	7-2
`blur_threshold_sec`	2초	같음	이보다 짧은 포커스 잃음은 이탈로 안 셈	7-3
`upload_retry_limit`	3번 (1·2·4초 간격)	같음	답변 음성·코드 올리기 재시도, 넘으면 장애	7-2
`stt_retry_limit` · 1번 시도 제한	2번 · 20초	2번 · 60초	음성 글자 바꾸기 재시도, 넘으면 장애	7-5
`caption_interval_sec`	13-2로 정함 (기준 1초)	3초	답변 중 임시 자막을 새로 만드는 간격. 그래픽카드가 바쁘면 그 차례는 건너뜀	5-2
`next_question_target_sec`	13-2로 정함 (기준 5초)	20초	답변 끝부터 다음 질문까지 목표(100번 중 95번). 꼬리질문 판단 시간도 이 값에서 남은 만큼만 줌	5-3, 7-5, 12장
`followup_min_sec`	13-2로 정함	같은 방법으로 잼	꼬리질문 판단에 줄 수 있는 남은 시간 이 이보다 짧으면 판단 없이 생략	7-5
`api_p95_target_sec`	13-2로 정함 (기준 0.3초)	확인 안 함	일반 화면 요청 응답 시간 목표(100번 중 95번)	12장
질문 만들기				

`question_regen_limit`	3번	같음	질문 자리마다 다시 만들기 최대 횟수	7-4 ⑤
`rag_example_count`	3개	같음	질문을 새로 만들 때 지시문에 예시로 넣는 비슷한 질문 수	7-4 ③
`question_attempt_timeout_sec`	8초	30초	다시 만들기 1번에 주는 시간	7-4 ⑤
`question_compose_budget_sec`	60초	180초	지원자 1명 질문 전체 만들기 제한. 시작 버튼 뒤 '질문 준비 중' 기다림의 상한이기도 함	7-4 ⑤⑥
`compose_slots`	1 (13-2로 정함)	1	LLM에서 질문 만들기에 쓰는 동시 요청 자리 수(그래픽카드 계산에 들어감)	6-2, 7-4 ⑥
이력서·OCR				
`cookie_secure`	끔 (이 PC 안에서 http로 실행)	같음	로그인 쿠키를 HTTPS로만 보낼지. 다른 PC에서 HTTPS로 접속하게 바꾸면 켬	11장
`upload_max_mb`	50MB (사용자에게 안 보임)	같음	서버 보호용 요청 크기 한도	7-6 ①
`ocr_sec_per_page`	13-2로 정함 (기준 8초)	잰 값	예상 처리 시간 계산용	7-6 ①
`resume_llm_sec`	13-2로 정함, 그 뒤 최근 평균으로 맞춤	잰 값	예상 처리 시간 중 LLM 정리 부분	7-6 ①
`ocr_fail_factor` · 최소	예상 시간의 2배 · 60초	같음	넘으면 불러오기 실패	7-6 ③
`ocr_fail_conf` · `ocr_low_conf`	0.5 · 0.80	같음	실패로 보는 평균 신뢰도 · 강조 표시 기준	7-6 ③④
채점·결정				
`quote_regen_limit` · `rejudge_limit`	1번 · 1번	같음	근거 틀림 때 다시 받기 · 결정 검증 실패 때 다시 정하기	8-2
`analysis_retry_limit` · 간격	3번 · 1·2·4분	같음	채점 작업 재시도	8-3
`needs_review_after_min`	30분	120분	이 시간을 넘으면 '확인 필요'	8-3
`stuck_scan_interval_min`	5분	같음	빠지거나 멈춘 작업 다시 넣기 주기	8-3, 10장
코딩테스트				
실행 시간 · 메모리 · CPU · 프로세스	10초 · 256MB · 1개 · 64	같음	문제당 실행 상자 제한(문제 화면에 표시)	
자동 저장	30초 (PRD COD-03)	같음		
계정·보안				

로그인 유지 · 로그인 잠금 · 재설정 링크	2시간(면접 중 늘림) · 5번/10분 · 30분 (PRD ACC)	같음		11장
`email_verification_required`	false (기능을 붙인 뒤 true)	false	이메일 인증 요구 여부	10장
자동 작업·알림				
`timezone`	Asia/Seoul	같음	모든 예약 작업과 공개·삭제일 계산 기준	10장
결과 공개 · 보관 기간 삭제	매일 10:00 · 매일 03:00	손으로 실행		10장
`mail_retry_limit` · 간격	3번 · 60·300·1800초	같음	메일 다시 보내기	10장
`smtp_host` · `smtp_port` · `mail_from`	smtp.gmail.com · 587 · 팀 Gmail 주소	Mailpit(mailpit · 1025)	메일 보내는 곳. 앱 비밀번호 (`smtp_password`)는 .env에만	2-1, 3장, 11장
`mail_daily_limit`	Gmail 계정 한도에 맞춤(약 500통)	없음	하루에 보낼 수 있는 양. 넘으면 다음 날로 미루고 관리자 대시보드에 표시	10장

개발 환경 값은 `.env.dev` 파일에만 두고 운영 설정 파일과 섞지 않는다. 운영 서버 값을 바꾸면 이 표와 개정 이력을 같이 고친다.

15. 확인 필요 사항

#	항목	지금 기본값	어떻게 정하나
T-01	모델 배치 (면접 중 4B 그래픽카드 + 채점 8B CPU)	6-2 기본안	13-2에서 재 보기
T-02	동시 면접 상한 N의 처음 값	2 (임시)	13-2에서 재 보기
T-03	한국어 TTS	Supertonic 후보	한국어 음질·속도를 들어 보고, 안 되면 다른 설치형 TTS
T-04	AI 결정이 기준 비교와 다르게 나오는 빈도	1번 다시 정한 뒤 확인 필요	정답 세트로 얼마나 자주 다른지 재고, 잦으면 지시문 고치기
T-05	예비 질문	종류별 5개 이상(이력서 자리용 '경험·프로젝트' 포함)	관리자가 미리 확인해서 넣기
T-06	OCR 엔진과 인식 품질	PaddleOCR 한국어, 신뢰도 기준 0.80	한국어 이력서 샘플로 정확도·처리 시간을 재고, 부족하면 Tesseract kor와 비교
T-07	추천안 3건 (PRD 추천안)	코딩테스트는 N에서 빼기·'다시 시도'(5-3), 이메일 인증은 새 지원부터(10장)	팀 확인
T-08	공고의 차별 표현 알아채기 (JOB-02, 팀원 미결 O-05)	미정 (추천: 금지·주의 표현 목록으로 찾고 경고만)	팀이 정한 뒤 5-1 공고 초안 API와 공고 편집 화면에 반영

부록. PRD 요구사항 추적표

PRD의 요구사항이 이 문서 어디에서 다뤄지는지 찾는 표다.

PRD	TRD 위치
ACC-01~06 (계정·가입)	4-1 users, 5-1, 10장 이메일 인증, 11장 로그인·권한
JOB-01~08 (공고·채점 기준)	4-1 job_postings·evaluation_criteria, 5-1
APP-01~07 (지원·준비)	4-1 applications·consents, 5-1, 2-2 frontend, 7-6 이력서·OCR
INT-01~10 (AI 면접)	5-2, 5-3, 7-1, 7-5, 10장 마감 처리, 14장
QST-01~05 (질문)	7-4
COD-01~07 (코딩테스트)	9장, 14장
EVL-01~08 (채점·결정)	8장, 14장
RPT-01~05 (결과 공개)	8-2 ⑥, 10장
ADM-01~08 (관리자 결과 관리)	5-1 /admin, 8-4, 10장
RES-01~06 (장애 이어가기)	7-1, 7-2
CHT-01~06 (화면 이탈)	7-3, 14장
AC-01	13-1
AC-23	13-1
10-1~10-3 (전제·제약·기능 외)	1-3, 6장, 11장, 12장, 14장
N7 · N8 (하지 않을 것)	1-3